

Chương 7:

QUẢN LÝ NHẬP XUẤT

ThS. GVC Tô Oai Hùng

1

1

Nội Dung

1. Nguyên lý phần cứng nhập/xuất.
2. Nguyên lý phần mềm nhập/xuất.
3. Đĩa cứng.

ThS. GVC Tô Oai Hùng

2

2

7.1 Nguyên Lý Phần Cứng Nhập/Xuất

1. Thiết bị nhập/xuất.
2. Bộ điều khiển thiết bị nhập/xuất.
3. Các thanh ghi nhập/xuất.
4. Kỹ thuật DMA.
5. Ngắt (*Interrupt*).

ThS. GVC Tô Oai Hùng

3

3

7.1.1 Thiết Bị Nhập/Xuất

❖ Thiết bị nhập/xuất (I/O) có thể chia làm hai loại chính:

- Thiết bị khối (*Block device*):
 - Lưu trữ thông tin theo các khối có kích thước cố định (512 ÷ 65,536 byte), mỗi khối có địa chỉ riêng.
 - Có thể đọc hoặc ghi từng khối một cách độc lập.
 - Đĩa cứng, USB, ...
- Thiết bị tuần tự (*Character device*):
 - Lưu trữ thông tin theo dãy tuần tự các bit, không có địa chỉ.
 - Thông tin được gửi/nhận một cách tuần tự.

ThS. GVC Tô Oai Hùng

4

4

Thiết Bị Nhập/Xuất

- Màn hình, bàn phím, máy in, card mạng, chuột.
- ❖ Một số thiết bị không thuộc hai loại trên: Clock, màn hình cảm ứng, ...

Thiết Bị Nhập/Xuất

- ❖ Đặc tính của các thiết bị I/O:
 - Tốc độ truyền dữ liệu.
 - Dung lượng lưu trữ, tốc độ truy xuất.
 - Chức năng.
 - Đơn vị truyền dữ liệu: Khối hay ký tự.
 - Biểu diễn dữ liệu.
 - Tình trạng lỗi: Nguyên nhân, cách báo lỗi, ...

Thiết Bị Nhập/Xuất

- ❖ Các thiết bị I/O có tốc độ giao tiếp rất khác nhau tùy vào tính chất sử dụng:

Device	Data rate	Device	Data rate
Keyboard	10 bytes/sec	FireWire 800	100 MB/sec
Mouse	100 bytes/sec	Gigabit Ethernet	125 MB/sec
56K modem	7 KB/sec	SATA 3 disk drive	600 MB/sec
Scanner at 300 dpi	1 MB/sec	USB 3.0	625 MB/sec
Digital camcorder	3.5 MB/sec	SCSI Ultra 5 bus	640 MB/sec
4x Blu-ray disc	18 MB/sec	Single-lane PCIe 3.0 bus	985 MB/sec
802.11n Wireless	37.5 MB/sec	Thunderbolt 2 bus	2.5 GB/sec
USB 2.0	60 MB/sec	SONET OC-768 network	5 GB/sec

ThS. GVC Tô Oai Hùng

7

7

7.1.2 Bộ Điều Khiển Thiết Bị Nhập/Xuất

- ❖ Các thiết bị I/O có thể giao tiếp với thiết bị khác.
- ❖ Để máy tính có thể giao tiếp với thiết bị I/O → Sử dụng bộ điều khiển thiết bị, gọi là device controller hay adapter (ở dạng chip).
- ❖ Mỗi bộ điều khiển thiết bị có một vài thanh ghi được sử dụng để giao tiếp với CPU.
- ❖ Bằng cách ghi vào các thanh ghi này, hệ điều hành có thể ra lệnh cho thiết bị cung cấp dữ liệu, chấp nhận dữ liệu, tự bật hoặc tắt hoặc thực hiện một số hành động, ...
- ❖ Bằng cách đọc từ các thanh ghi này, hệ điều hành có thể biết được trạng thái của thiết bị có sẵn sàng chấp nhận một lệnh mới hay không.

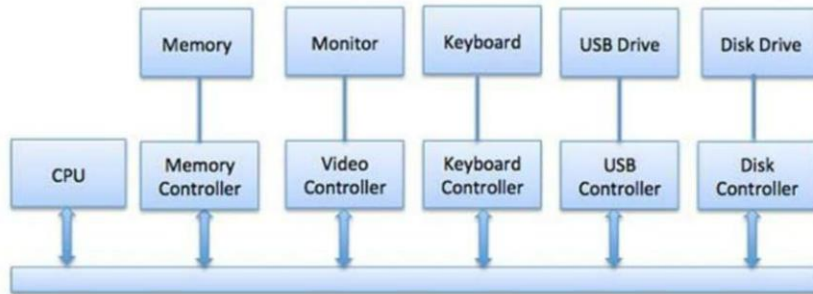
ThS. GVC Tô Oai Hùng

8

8

Bộ Điều Khiển Thiết Bị Nhập/Xuất

- ❖ Mô hình kết nối CPU, bộ nhớ, bộ điều khiển và thiết bị I/O.



ThS. GVC Tô Oai Hùng

9

9

7.1.3 Các Thanh Ghi Nhập/Xuất

- ❖ Thanh ghi lệnh: Thanh ghi chứa mã lệnh chức năng mà CPU ghi vào để controller thực hiện, chiều di chuyển thông tin của thanh ghi này là: CPU → controller (OUT).
- ❖ Thanh ghi trạng thái: Thanh ghi chứa các bit thông tin về trạng thái hiện hành của thiết bị I/O tương ứng (bận, rảnh,...), chiều di chuyển thông tin của thanh ghi này là: controller → CPU (IN).
- ❖ Thanh ghi dữ liệu xuất: Chứa dữ liệu mà CPU muốn xuất ra thiết bị I/O, chiều di chuyển thông tin của thanh ghi này là: CPU → controller (OUT).

ThS. GVC Tô Oai Hùng

10

10

Các Thanh Ghi Nhập/Xuất

- ❖ Thanh ghi dữ liệu nhập: Chứa dữ liệu mà thiết bị I/O đưa vào hệ thống, chiều di chuyển thông tin của thanh ghi này là: controller → CPU (IN).
- ❖ Mỗi thanh ghi cần có địa chỉ truy xuất duy nhất.
- ❖ Có ba loại địa chỉ:
 - Địa chỉ I/O.
 - Địa chỉ memory-mapped I/O.
 - Địa chỉ hỗn hợp.

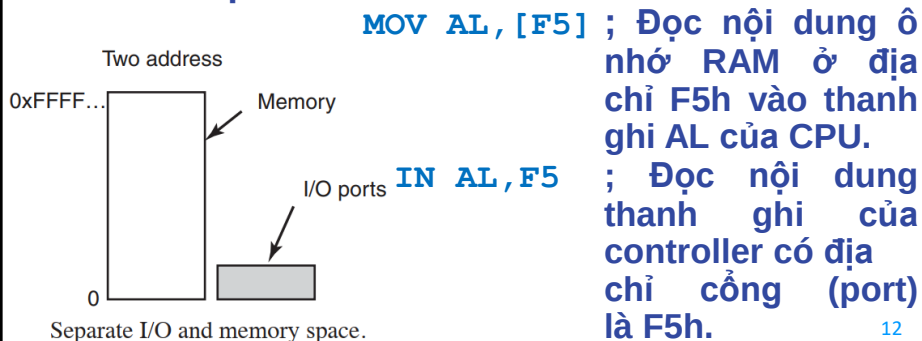
ThS. GVC Tô Oai Hùng

11

11

Các Thanh Ghi Nhập/Xuất

- ❖ Địa chỉ I/O có hai loại:
 - Địa chỉ ô nhớ dùng để truy xuất các ô nhớ trong RAM.
 - Địa chỉ I/O dùng để truy xuất các thanh ghi của các mạch controller.
 - Ví dụ:



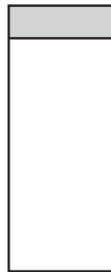
12

12

Các Thanh Ghi Nhập/Xuất

- ❖ Địa chỉ memory-mapped I/O:
 - Cách tiếp cận này được giới thiệu với máy PDP-11 (1960).
 - Ánh xạ tất cả các thanh ghi điều khiển vào không gian bộ nhớ.

One address space



ThS. GVC Tô Oai Hùng

Memory-mapped I/O.

13

13

Các Thanh Ghi Nhập/Xuất

- Mỗi thanh ghi điều khiển được gán một địa chỉ bộ nhớ duy nhất.
- Muốn truy xuất thanh ghi I/O, CPU sẽ dùng lệnh truy xuất ô nhớ như bình thường.
- Ví dụ:

`MOV AL, [F5]` ; Đọc nội dung thanh ghi I/O ở địa chỉ F5h, tùy vào địa chỉ này đang gán cho thanh ghi nào.

ThS. GVC Tô Oai Hùng

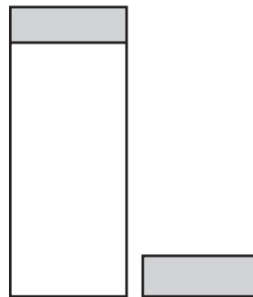
14

14

Các Thanh Ghi Nhập/Xuất

- ❖ Địa chỉ hỗn hợp (*hybrid*):
 - Kết hợp địa chỉ dạng memory-mapped I/O với địa chỉ (port) I/O riêng biệt cho các thanh ghi điều khiển.

Two address spaces



Hybrid.

ThS. GVC Tô Oai Hùng

15

15

Các Thanh Ghi Nhập/Xuất

- Địa chỉ ô nhớ dành để truy xuất các ô nhớ trong RAM.
- Địa chỉ I/O dành để truy xuất các thanh ghi của các mạch controller.
- Tùy theo tính chất sử dụng của từng thanh ghi → sử dụng địa chỉ (port) I/O hay địa chỉ memory với lệnh tương ứng để truy xuất.

ThS. GVC Tô Oai Hùng

16

16

7.1.4 Kỹ Thuật DMA

- ❖ CPU giao tiếp với I/O: Có 3 cách.
 - Dùng lệnh.
 - Dùng kỹ thuật Memory-mapped I/O.
 - Dùng kỹ thuật Direct memory access (DMA).

ThS. GVC Tô Oai Hùng

17

17

Kỹ Thuật DMA

- ❖ Dùng lệnh:
 - CPU phát sinh lệnh I/O (gửi dữ liệu hoặc đọc dữ liệu) đến các đơn vị I/O.
 - Sau đó, nó chờ cho đến khi thao tác này hoàn tất.

ThS. GVC Tô Oai Hùng

18

18

Kỹ Thuật DMA

- ❖ Kỹ thuật Memory-mapped I/O (không phải địa chỉ memory-mapped I/O):
 - Một không gian địa chỉ được chia sẻ giữa bộ nhớ và I/O. Thiết bị được kết nối trực tiếp với các vị trí trong bộ nhớ chính để chuyển dữ liệu đến bộ nhớ hoặc đọc dữ liệu từ bộ nhớ không cần thông qua CPU.
 - HĐH phân bổ bộ đệm trong bộ nhớ và báo cho I/O dùng bộ đệm đó để gửi dữ liệu đến CPU. Thiết bị I/O hoạt động không đồng bộ với CPU, ngắt CPU khi kết thúc.
 - Ưu điểm: Các lệnh có thể truy cập bộ nhớ đều có thể được sử dụng để thao tác với thiết bị I/O → nhanh → được sử dụng cho các thiết bị I/O tốc độ cao.

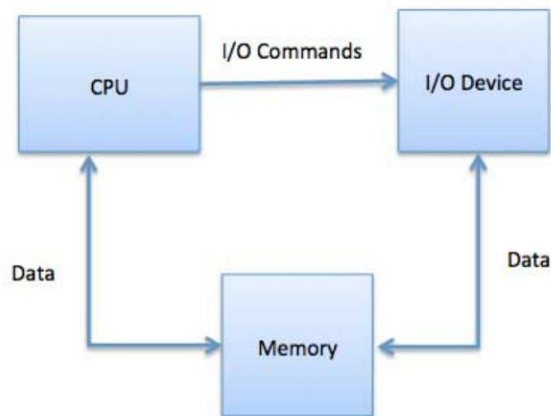
ThS. GVC Tô Oai Hùng

19

19

Kỹ Thuật DMA

- ❖ Kỹ thuật Memory-mapped I/O (tt):



ThS. GVC Tô Oai Hùng

20

20

Kỹ Thuật DMA

- ❖ Kỹ thuật Direct memory access (DMA):
 - Việc chuyển thông tin giữa thiết bị I/O với bộ nhớ máy tính được thực hiện thông qua CPU theo cơ chế tuần tự.
 - Mỗi lần cần di chuyển dữ liệu từ thiết bị I/O (đang nằm trong thanh ghi của controller), CPU phải thực hiện 2 lệnh máy liên tiếp:
 1. Lệnh IN (hay MOV) để di chuyển dữ liệu từ controller vào thanh ghi của CPU.
 2. Lệnh MOV để di chuyển dữ liệu từ thanh ghi CPU ra ô nhớ RAM xác định.
 - Hạn chế: Không hiệu quả.

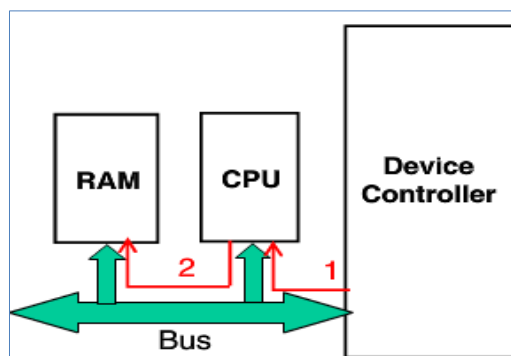
ThS. GVC Tô Oai Hùng

21

21

Kỹ Thuật DMA

- Tốn nhiều chu kỳ để di chuyển dữ liệu.
- Dữ liệu phải đi qua trung gian là CPU.
- → Sử dụng kỹ thuật DMA.



ThS. GVC Tô Oai Hùng

22

22

Kỹ Thuật DMA

❖ Kỹ thuật Direct memory access (tt):

- Truy cập bộ nhớ trực tiếp (DMA) nghĩa là CPU cho phép đơn vị I/O đọc hoặc ghi vào bộ nhớ (RAM) mà không thông qua CPU.
- DMA tự điều khiển việc trao đổi dữ liệu giữa bộ nhớ chính và thiết bị I/O.
- CPU chỉ tham gia vào lúc bắt đầu và kết thúc quá trình chuyển dữ liệu và chỉ bị ngắt sau khi toàn bộ khối dữ liệu đã được chuyển đi.
- CPU khởi động các thanh ghi của DMA, các thanh ghi này chứa địa chỉ ô nhớ và số byte cần chuyển.

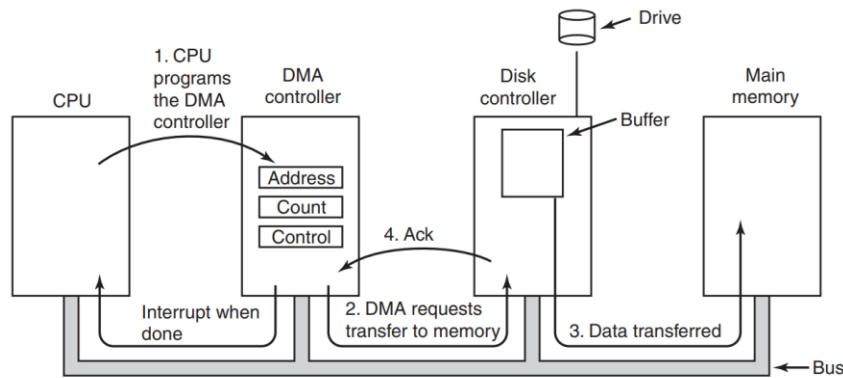
ThS. GVC Tô Oai Hùng

23

23

Kỹ Thuật DMA

- DMA chuyển số liệu và khi chấm dứt thì trả quyền điều khiển cho CPU: Bộ điều khiển DMA báo (ngắt) CPU khi hoàn thành công việc theo từng block dữ liệu.



Operation of a DMA transfer.

24

Kỹ Thuật DMA

- Sau bước 4, bộ điều khiển DMA tăng địa chỉ bộ nhớ và giảm số byte cần chuyển (thanh ghi count).
- Nếu số byte vẫn lớn hơn 0, các bước từ 2 đến 4 được lặp lại cho đến khi giá trị của count bằng 0.
- Vào thời điểm đó, bộ điều khiển DMA ngắt CPU để báo rằng quá trình chuyển dữ liệu đã hoàn tất.

ThS. GVC Tô Oai Hùng

25

25

Kỹ Thuật DMA

- ❖ Lưu ý: Có sự khác nhau giữa kỹ thuật Memory-mapped I/O với DMA:
 - Memory-mapped I/O cho phép CPU điều khiển phần cứng bằng cách đọc và ghi các địa chỉ bộ nhớ cụ thể.
 - DMA cho phép phần cứng đọc và ghi trực tiếp bộ nhớ mà không cần đến CPU.

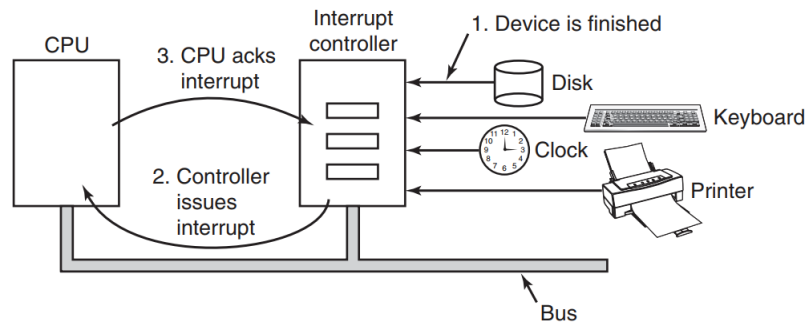
ThS. GVC Tô Oai Hùng

26

26

7.1.5 Kỹ Thuật Ngắt

- ❖ Bộ điều khiển thiết bị sẽ báo (ngắt) cho CPU khi hoàn thành công việc theo từng đơn vị dữ liệu → CPU không phải chờ trong khi thiết bị I/O thực hiện công việc.



How an interrupt happens. The connections between the devices and the controller actually use interrupt lines on the bus rather than dedicated wires.

27

27

7.2 Nguyên Lý Phần Mềm Nhập/Xuất

1. Mục tiêu của phần mềm nhập/xuất.
2. Nhập/xuất được lập trình.
3. Cơ chế ngắt.
4. Kỹ thuật DMA.

28

7.2.1 Mục Tiêu Của Phần Mềm Nhập/Xuất

- ❖ **Độc lập thiết bị:** Chương trình ứng dụng có thể truy cập bất kỳ thiết bị I/O nào không phụ thuộc vào tính chất vật lý hay loại thiết bị.
- ❖ **Đặt tên thiết bị đồng nhất:** Tên thiết bị là một chuỗi hay một số nguyên giống như tên một file và không phụ thuộc vào thiết bị nào.
- ❖ **Che dấu việc xử lý lỗi:** Nếu có lỗi trong khi truy xuất I/O, hệ thống phần mềm I/O phải cố gắng xử lý ở cấp thấp nhất. Nếu không được thì trình điều khiển thiết bị xử lý (cố gắng đọc lại khối). Chỉ khi các lớp dưới không thể giải quyết được vấn đề thì các lớp trên mới được thông báo về nó.

ThS. GVC Tô Oai Hùng

29

29

Mục Tiêu Của Phần Mềm Nhập/Xuất

- ❖ **Chuyển đổi cách nhập/xuất dữ liệu dạng bất đồng bộ (không chờ nhập/xuất) thành dạng đồng bộ (chờ nhập/xuất):**
 - Ở cấp vật lý: Mỗi lần cần đọc một sector đĩa, CPU sẽ xuất lệnh đọc ra controller, chờ ngắt khi có dữ liệu, đọc khối dữ liệu vào bộ nhớ.
 - Ở cấp ứng dụng: Cung cấp một hàm chức năng, ứng dụng gọi hàm này → trả về kết quả → ứng dụng có dữ liệu để xử lý tiếp.
- ❖ **Sử dụng bộ đệm dữ liệu nhập/xuất:**
 - Dữ liệu từ một thiết bị I/O đưa vào máy tính cần được cache lại để cung cấp cho ứng dụng khi có yêu cầu.

ThS. GVC Tô Oai Hùng

30

30

Mục Tiêu Của Phần Mềm Nhập/Xuất

- ❖ Thiết bị dùng chung (*shared*):
 - Tại mỗi thời điểm có thể phục vụ cho nhiều ứng dụng đồng thời.
- ❖ Thiết bị dùng riêng (*dedicated*):
 - Chỉ phục vụ một ứng dụng tại mỗi thời điểm.

ThS. GVC Tô Oai Hùng

31

31

7.2.2 Nhập/Xuất Được Lập Trình

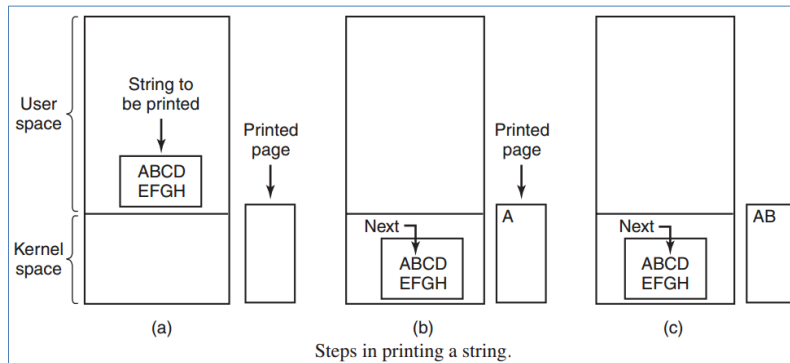
- ❖ Dạng nhập/xuất đơn giản nhất là để CPU thực hiện tất cả công việc.
- ❖ Phương pháp này được gọi là nhập/xuất được lập trình (*Programmed I/O*).
- ❖ Xét một quy trình người dùng muốn in chuỗi tám ký tự "ABCDEFGH" trên máy in thông qua giao diện nối tiếp.

ThS. GVC Tô Oai Hùng

32

32

Nhập/Xuất Được Lập Trình



```
copy_from_user(buffer, p, count);          /* p is the kernel buffer */
for (i = 0; i < count; i++) {               /* loop on every character */
    while (*printer_status_reg != READY);    /* loop until ready */
    *printer_data_register = p[i];          /* output one character */
}
return_to_user();
```

T

Writing a string to the printer using programmed I/O.

33

Nhập/Xuất Được Lập Trình

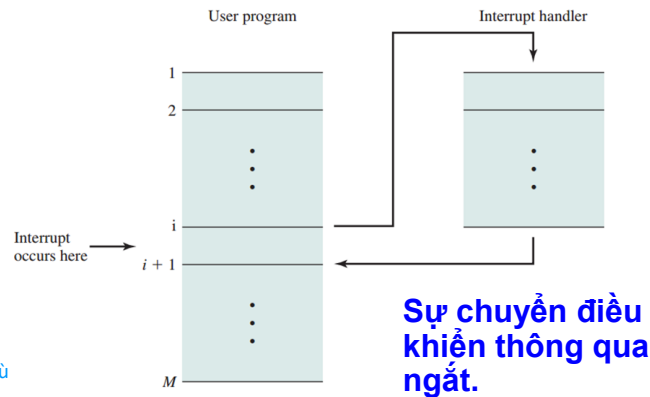
- ❖ Trong đoạn mã trên, sau khi nhận chuỗi ký tự cần in, HĐH sẽ lặp kiểm tra trạng thái máy in, nếu rảnh thì xuất ra 1 ký tự cho đến hết chuỗi.
- ❖ Nhận xét:
 - Đơn giản.
 - Do CPU thực hiện hoàn toàn → Chiếm dụng CPU toàn thời gian cho đến khi tất cả I/O hoàn thành.
 - Không hiệu quả trong các hệ thống phức tạp.

ThS. GVC Tô Oai Hùng

34

7.2.3 Cơ Chế Ngắt

- ❖ Ngắt (*Interrupt*) là dừng chương trình đang thực thi để thực thi chương trình khác.
- ❖ Ngắt được chia thành 2 loại: Ngắt cứng và ngắt mềm.
- ❖ Ví dụ ngắt mềm:



35

Cơ Chế Ngắt

- ❖ Để tận dụng CPU, khi thiết bị I/O làm việc, CPU thực hiện công việc khác → Sử dụng cơ chế ngắt (*Interrupt-Driven I/O*).
- ❖ Mã lệnh lời gọi hệ thống được hiển thị như sau:

Print system call:

```
copy_from_user(buffer, p, count);
```

```
enable_interrupts();
```

```
/* loop until printer controller becomes READY */
```

```
while (*printer_status_reg != READY) ;
```

```
/* send to the first character to the printer controller */
```

```
*printer_data_register = p[0];
```

```
scheduler(); // do some other task
```

Code executed at the time the print system call is made.

ThS. GVC Tô Oai Hùng

36

36

Cơ Chế Ngắt

- ❖ Khi lời gọi hệ thống (*system call*) để in chuỗi được thực hiện, bộ đệm được sao chép vào không gian hệ điều hành.
- ❖ CPU sẽ khởi động chế độ phục vụ ngắt từ máy in và không đợi cho đến khi máy in sẵn sàng.
- ❖ Thay vào đó hàm `scheduler()` được gọi và tiến trình khác thực thi.
 - Tiến trình hiện hành được đặt vào trạng thái chờ (*blocked*).
- ❖ Ký tự đầu tiên được sao chép vào máy in.
- ❖ Khi in xong một ký tự và sẵn sàng nhận ký tự tiếp theo, máy in sinh ra một ngắt cứng.
- ❖ Ngắt này làm dừng tiến trình hiện tại và lưu trạng thái của tiến trình đó.

ThS. GVC Tô Oai Hùng

37

37

Cơ Chế Ngắt

- ❖ Thủ tục phục vụ ngắt (*interrupt-service procedure*) của máy in được thực thi:

Interrupt service procedure:

```

if (count == 0) { /* we are finishes printing the string */
    unblock user(); /* unblock the user process
                    that wanted to print the string */
} else {
    /* copy one more character to the controller buffer */
    *printer_data_register = p[i];
    count = count - 1;
    i = i + 1;
}
acknowledge_interrupt();
return_from_interrupt(); /* finished serving the
                          interrupt */

```

Interrupt service procedure for the printer.

ThS. GVC Tô Oai Hùng

38

38

Cơ Chế Ngắt

- ❖ Thủ tục này chặn những yêu cầu đến chuỗi cần in cho đến khi toàn bộ chuỗi được in.
- ❖ Nếu không còn ký tự nào để in thì trình xử lý ngắt thực hiện bỏ chặn người dùng.
- ❖ Ngược lại, nó xuất ký tự tiếp theo, xác nhận ngắt và quay lại tiến trình vừa chạy trước khi ngắt, tiếp tục từ nơi nó dừng lại.

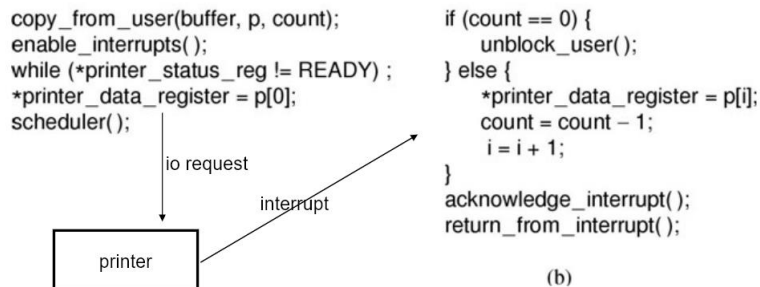
ThS. GVC Tô Oai Hùng

39

39

Cơ Chế Ngắt

❖ Tóm tắt cơ chế ngắt:



- Lờn gọi hệ thống kết thúc bằng việc gọi scheduler (a).
- Thiết bị sẽ thực hiện ngắt khi nó sẵn sàng.
- Ngắt được xử lý như (b).

ThS. GVC Tô Oai Hùng

40

40

7.2.4 Kỹ Thuật DMA

- ❖ Một nhược điểm rõ ràng của việc sử dụng cơ chế ngắt (*Interrupt-Driven I/O*) là ngắt xảy ra trên mỗi ký tự.
- ❖ Ngắt chiếm thời gian, vì thế phương pháp này sẽ làm lãng phí thời gian của CPU.
- ❖ Một giải pháp là sử dụng DMA.
- ❖ Ý tưởng ở đây là để bộ điều khiển DMA đưa các ký tự vào máy in cùng một lúc mà không cần đến CPU.
- ❖ Về bản chất, DMA là nhập/xuất được lập trình.
- ❖ Chỉ với bộ điều khiển DMA, nó sẽ thực hiện tất cả công việc thay vì CPU.

ThS. GVC Tô Oai Hùng

41

41

Kỹ Thuật DMA

- ❖ Chiến lược này yêu cầu phần cứng đặc biệt (DMA controller) để CPU thực hiện công việc khác trong quá trình nhập/xuất.
- ❖ Mã lệnh in một chuỗi dùng DMA như sau:

Print system call

```
copy_from_user(buffer, p, count);
set_up_DMA_controller();
scheduler();
```

Code executed when the print system call is made.

Interrupt service procedure

```
acknowledge_interrupt();
unblock_user();
return_from_interrupt();
```

Interrupt-service procedure.

ThS. GVC Tô Oai Hùng

42

42

Kỹ Thuật DMA

- ❖ Lợi ích lớn nhất của DMA là giảm số lần ngắt từ một lần cho mỗi lần ký tự thành một lần cho mỗi bộ đệm được in.

ThS. GVC Tô Oai Hùng

43

43

7.3 Đĩa Cứng

1. Giới thiệu.
2. Định dạng đĩa.
3. Các thuật toán điều phối đĩa.

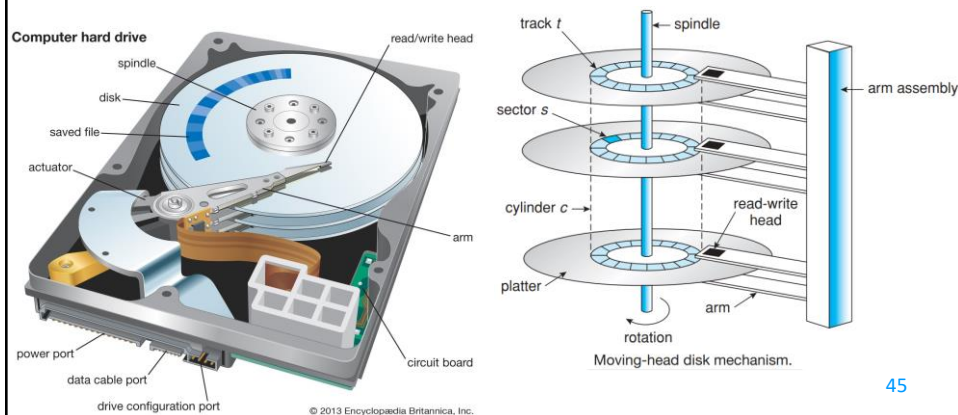
ThS. GVC Tô Oai Hùng

44

44

7.3.1 Giới Thiệu

- ❖ **Đĩa cứng (hard disk) là đĩa từ (magnetic disk) có giá thành rẻ hơn RAM trên mỗi bit lưu trữ.**
- ❖ **Cấu trúc đĩa cứng:**



45

Giới Thiệu

- ❖ **Đĩa vật lý là không gian dữ liệu 3 chiều:**
 - Gồm nhiều cylinder.
 - Mỗi cylinder gồm nhiều track.
 - Mỗi track chứa nhiều sector.
- ❖ **Sector là đơn vị lưu trữ thông tin nhỏ nhất ở cấp vật lý (không thể truy xuất từng byte dữ liệu trên đĩa).**
- ❖ **Để truy xuất 1 sector cần phải chỉ ra vị trí của sector đó.**
- ❖ **Vị trí sector được thể hiện bằng 3 thông số:**
 - Chỉ số sector, track và head.
 - Track được đánh số theo thứ tự từ ngoài vào, bắt đầu từ 0.

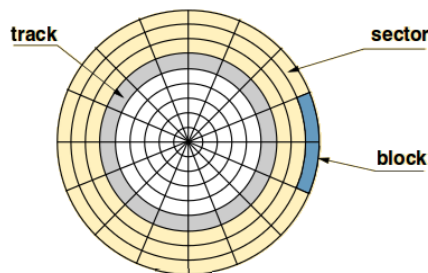
ThS. GVC Tô Oai Hùng

46

46

Giới Thiệu

- Head được đánh số từ trên xuống, bắt đầu từ 0.
- Sector được đánh số bắt đầu từ 1 theo chiều ngược với chiều quay của đĩa.
- ❖ Muốn truy xuất một sector, cần xác định tọa độ 3 chiều của nó (C, H, S) – Cylinder, Head, Sector.



ThS. GVC Tô Oai Hùng

47

47

Giới Thiệu

- ❖ Đĩa SLED (Single Large Expensive Disk):
 - Đĩa đơn có tốc độ truy xuất chậm.
 - Độ tin cậy thấp.
 - → Khắc phục bằng kỹ thuật RAID (Hệ thống đĩa dự phòng - *Redundant Array of Inexpensive Disks*).
- ❖ RAID:
 - Kết nối một dãy các ổ cứng có chi phí thấp lại với nhau để hình thành một đĩa logic có dung lượng lớn.
 - Có 7 dạng RAID khác nhau.
 - Ưu điểm của RAID:
 - Dự phòng.
 - Hiệu quả cao.
 - Giá thành thấp.

ThS. GVC Tô Oai Hùng

48

48

Đĩa Cứng RAID 0

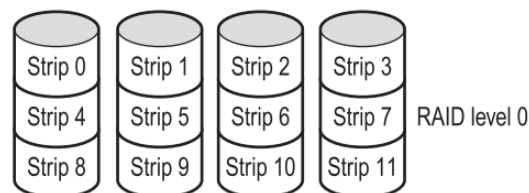
- ❖ Dùng kỹ thuật striping: Gộp k sector thành một strip, dùng n đĩa để hiện thực đĩa luận lý như sau:
 - Phân bổ các strip theo kiểu round-robin.
 - Nếu máy tính truy xuất khối dữ liệu dài n strip, controller trên RAID 0 sẽ biết được khối dữ liệu n strip này nằm trên n đĩa thô khác nhau.
 - Nó sẽ ra lệnh n đĩa đồng thời, mỗi đĩa truy xuất 1 strip.
 - Dữ liệu truy xuất sẽ được hợp lại/phân ra bởi controller.

ThS. GVC Tô Oai Hùng

49

49

Đĩa Cứng RAID 0



- ❖ Ưu điểm:
 - Tăng tốc độ truy xuất lên n lần.
 - Tăng hiệu quả lưu trữ.
 - Không làm mất dung lượng dữ liệu.
- ❖ Nhược điểm
 - Không có ổ dự phòng.
 - Giảm độ tin cậy n lần.

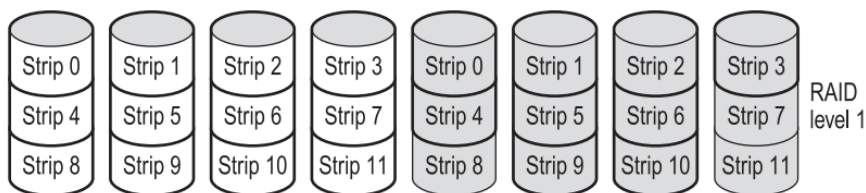
ThS. GVC Tô Oai Hùng

50

50

Đĩa Cứng RAID 1

- ❖ Dùng kỹ thuật Master/Mirror: Nhân bản dữ liệu.
- ❖ Ngoài n đĩa có sẵn (master), dùng thêm n đĩa nữa (slave).
- ❖ Mỗi khi ghi dữ liệu, controller sẽ ghi dữ liệu đồng thời lên cả master lẫn slave.
- ❖ Mỗi khi đọc dữ liệu, controller chỉ cần yêu cầu master thực hiện.



ThS. GVC Tô Oai Hùng

51

51

Đĩa Cứng RAID 1

- ❖ Trường hợp master trục trặc, controller đổi vai trò của master và slave → Thực hiện lại việc đọc dữ liệu.
- ❖ Phục hồi: Sao chép lại dữ liệu từ đĩa không có lỗi.
- ❖ Ưu điểm: Khả năng dự phòng dữ liệu.
- ❖ Nhược điểm:
 - Sử dụng nhiều đĩa cứng → Hao phí. Để khắc phục nhược điểm này, sử dụng RAID 2.
 - Không tăng hiệu suất thực thi.
 - Tốn thời gian thay đổi ổ hoạt động khi có sự cố.

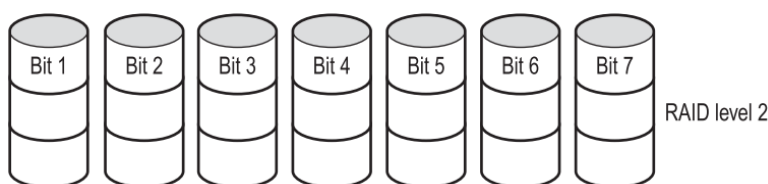
ThS. GVC Tô Oai Hùng

52

52

Đĩa Cứng RAID 2

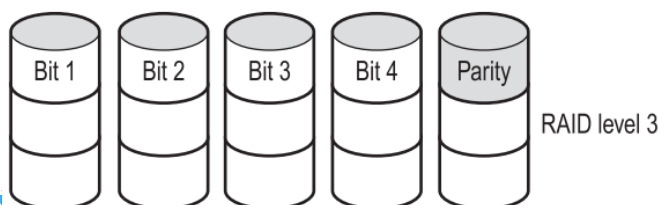
- ❖ Để lưu 32 bit dữ liệu, controller sẽ đổi 32 bit dữ liệu thành mã sửa sai Hamming 39 bit, từng bit sẽ được ghi lên một đĩa khác nhau.
- ❖ Tốc độ: Tăng lên 32 lần so với SLED.
- ❖ Độ tin cậy: Nếu có lỗi ở một đĩa, nhờ mã Hamming, controller sẽ tự sửa sai để xác định 32 bit dữ liệu gốc.
- ❖ 7 bit Hamming được mã hóa trên bảy ổ đĩa, một bit trên mỗi ổ đĩa.



53

Đĩa Cứng RAID 3

- ❖ RAID 3 là phiên bản đơn giản của RAID 2.
- ❖ Một bit chẵn lẻ được tính cho mỗi từ dữ liệu và được ghi vào một ổ đĩa chẵn lẻ (parity).
- ❖ Tốc độ truy xuất tập tin kích thước nhỏ rất nhanh.
- ❖ Độ tin cậy: Nếu có lỗi ở một đĩa, nhờ bit parity phát hiện lỗi và tín hiệu báo lỗi ở đĩa thứ i, controller sẽ tự sửa sai bit i để xác định dữ liệu gốc.



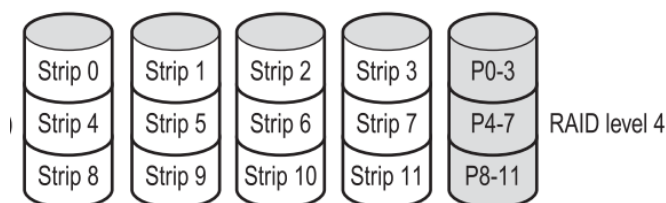
ThS. GVC Tô Oai H

54

54

Đĩa Cứng RAID 4

- ❖ Tương tự như RAID 0.
- ❖ Dữ liệu ghi trên n đĩa và 1 đĩa chứa bit phát hiện lỗi (parity).
- ❖ Độ tin cậy: Nếu có lỗi ở một đĩa, nhờ bit parity phát hiện lỗi và tín hiệu báo lỗi ở đĩa thứ i.



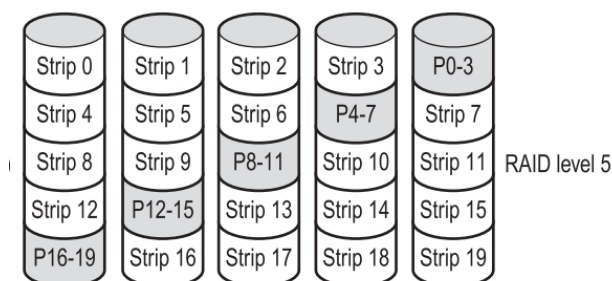
ThS. GVC Tô Oai Hùng

55

55

Đĩa Cứng RAID 5

- ❖ Sử dụng kỹ thuật striping theo từng Block và phân chia thông tin "parity" (chẵn lẻ).
- ❖ Cần ít nhất 3 đĩa.
- ❖ Dữ liệu ghi vào n đĩa + 1 đĩa ghi bit parity dự phòng theo kiểu luân phiên.



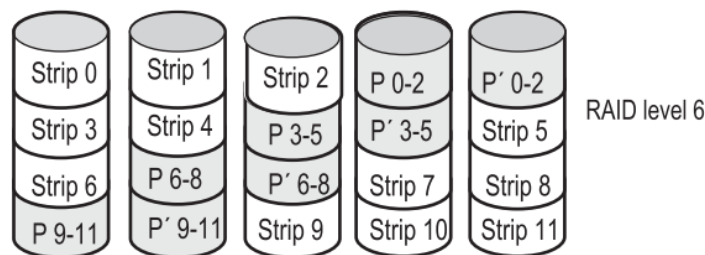
ThS. GVC Tô Oai Hùng

56

56

Đĩa Cứng RAID 6

- ❖ Phát triển từ RAID 5.
- ❖ Cần ít nhất 4 đĩa.
- ❖ Dữ liệu ghi vào n đĩa + 2 đĩa ghi bit parity dự phòng chặn/lỗi độc lập với nhau theo kiểu luân phiên.



ThS. GVC Tô Oai Hùng

57

57

7.3.2 Định Dạng Đĩa

- ❖ Trước khi đĩa có thể được sử dụng, mỗi mặt đĩa phải được định dạng cấp thấp do phần mềm thực hiện.
- ❖ Đĩa được định dạng tạo thành hàng loạt các track, mỗi track chứa các sector.
- ❖ Mọi sector bao gồm các trường:
 - Preamble: Bắt đầu với một số bit nhất định dùng để xác định vị trí bắt đầu của sector, cylinder number, sector number và các thông tin khác.
 - Data: Dữ liệu, kích thước được xác định bởi chương trình định dạng cấp thấp, thường là 512 byte.

ThS. GVC Tô Oai Hùng

58

58

7.3.2 Định Dạng Đĩa

- ECC: Chứa thông tin được sử dụng để khôi phục lỗi, kích thước và nội dung của trường này phụ thuộc vào nhà sản xuất.



A disk sector.

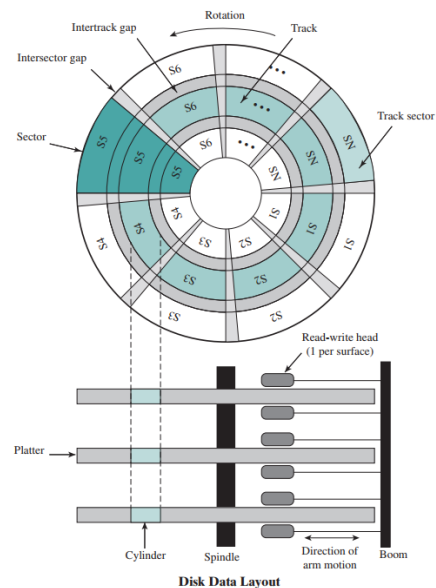
ThS. GVC Tô Oai Hùng

59

59

Định Dạng Đĩa

- ❖ Sau khi định dạng mức thấp, dung lượng đĩa bị giảm, thường thấp hơn 20% so với dung lượng chưa được định dạng.

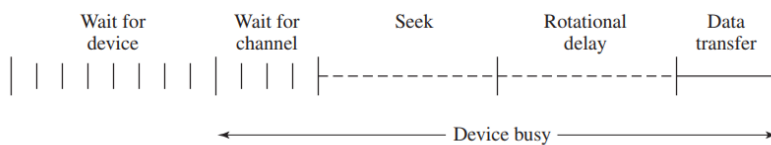


ThS. GVC Tô Oai Hùng

60

7.3.3 Các Thuật Toán Điều Phối Đĩa

- ❖ Tốc độ đọc/ghi đĩa phụ thuộc ba yếu tố:
 - Seek time: Thời gian di chuyển đầu đọc tới track hay cylinder thích hợp.
 - Rotational delay (rotational latency): Thời gian quay đĩa đến sector cần đọc.
 - Actual data transfer time: Thời gian truyền dữ liệu thực tế.



Timing of a Disk I/O Transfer

ThS. GVC Tô Oai Hùng

61

61

Các Thuật Toán Điều Phối Đĩa

- ❖ Thời gian truyền: $T = b/rN$
 - T = Thời gian truyền.
 - b = Số byte được truyền.
 - N = Số byte trên 1 track.
 - r = Số vòng quay mỗi giây.
- ❖ Vì vậy, tổng thời gian truy xuất trung bình:

$$T_a = T_s + 1/2r + b/rN$$
 Trong đó:
 - T_s = Thời gian tìm kiếm trung bình (trên các đĩa cứng hiện tại $T_s < 10$ ms).

ThS. GVC Tô Oai Hùng

62

62

Các Thuật Toán Điều Phối Đĩa

- ❖ Các thuật toán điều phối đĩa được dùng để xác định thứ tự di chuyển giữa các track trên đĩa một cách tối ưu.
- ❖ Các thuật toán điển hình:
 - FCFS (First-come, first-served).
 - SSTF (Shortest-seek-time-first)
 - SCAN (Elevator algorithm)
 - C-SCAN (One-way elevator)
 - LOOK.
 - C-LOOK.

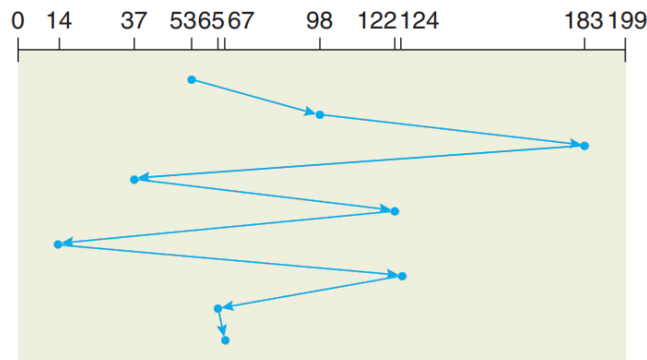
ThS. GVC Tô Oai Hùng

63

63

First-Come First-Served (FCFS)

- ❖ Di chuyển theo thứ tự truy xuất.
- ❖ Ví dụ: Truy xuất các cylinder 98, 183, 37, 122, 14, 124, 65, 67 và vị trí hiện tại của đầu đọc ở cylinder 53:



ThS. GVC Tô Oai Hùng

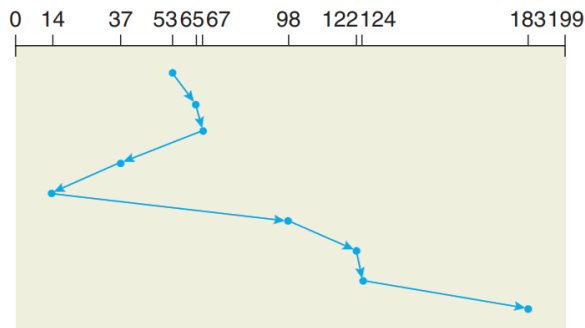
FCFS disk scheduling.

64

64

Shortest-Seek-Time-First (SSTF)

- ❖ Truy xuất nào có di chuyển ít nhất thì thực hiện trước.
- ❖ Ví dụ: Truy xuất các cylinder 98, 183, 37, 122, 14, 124, 65, 67 và vị trí hiện tại của đầu đọc ở cylinder 53:



ThS. GVC Tô Oai Hùng

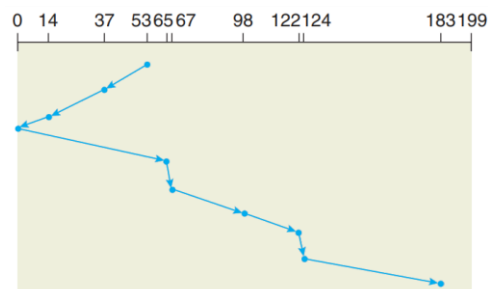
SSTF disk scheduling.

65

65

SCAN (Elevator Algorithm)

- ❖ Đầu đọc di chuyển từ 1 phía. Khi đến cylinder cuối cùng phía đó mới quay lại.
- ❖ Trong khi quay về, sẽ truy xuất cylinder nào được yêu cầu.
- ❖ Ví dụ: Truy xuất các cylinder 98, 183, 37, 122, 14, 124, 65, 67 và vị trí hiện tại của đầu đọc ở cylinder 53:



ThS. GVC Tô Oai Hùng

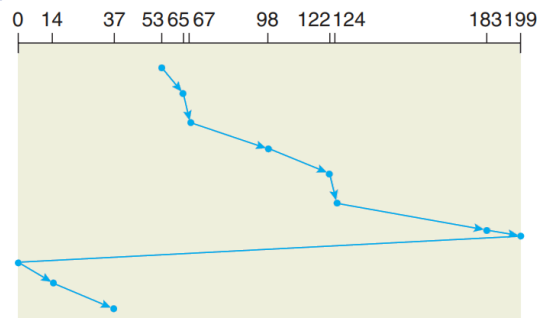
SCAN disk scheduling.

66

66

C-SCAN (Circular SCAN)

- ❖ C-SCAN (One-way elevator) là một biến thể của SCAN nhưng chỉ truy xuất cylinder theo 1 chiều.
- ❖ Khi đến cylinder cuối cùng, đầu đọc quay về vị trí bắt đầu 0. Trong khi quay về, không phục vụ yêu cầu nào (1 chiều).



ThS. GVC Tô Oai Hùng

C-SCAN disk scheduling.

67

67

LOOK

- ❖ SCAN và C-SCAN đều di chuyển đầu đọc qua toàn bộ chiều rộng của đĩa.
- ❖ Trong thực tế, không có thuật toán nào được thực hiện theo cách này.
- ❖ Thông thường, đầu đọc chỉ thực hiện khi có yêu cầu cuối cùng trên mỗi hướng.
- ❖ Sau đó, nó quay lại mà không đi đến track cuối của đĩa.
- ❖ Phiên bản SCAN thực hiện theo cách này được gọi là LOOK, vì chúng tìm kiếm (*look for*) yêu cầu trước tiếp tục di chuyển theo hướng đã cho.
- ❖ Ví dụ: Truy xuất các cylinder 98, 183, 37, 122, 14, 124, 65, 67 và vị trí hiện tại của đầu đọc ở cylinder 53 (xem như bài tập).

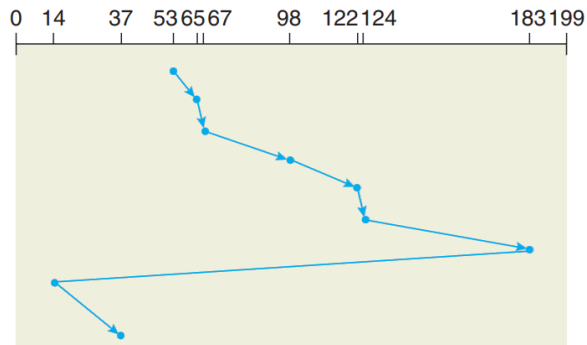
ThS. GVC Tô Oai Hùng

68

68

C-LOOK

- ❖ C-LOOK với LOOK tương tự như C-SCAN với SCAN.
- ❖ Ví dụ: Truy xuất các cylinder 98, 183, 37, 122, 14, 124, 65, 67 và vị trí hiện tại của đầu đọc ở cylinder 53:



C-LOOK disk scheduling.

69